Save Point

Setup guide: Convex, Clerk, Stripe and Anthropic

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

These are the steps that need a person with accounts. Until they're done, the backend runs on a local anonymous Convex deployment and all its tests pass with fakes (see `backend.md`). Do them in order: Convex first, because Clerk and Stripe both need its URLs.

This guide covers the backend (Phase 2), the Flutter app (Phase 3) and the web app (Phase 4).

---

1. Convex

1. Create an account at convex.dev.
2. In `backend/`, run `npx convex dev`. Choose to create a new project named `quillfin`. This writes `backend/.env.local`, which is ignored by git, and replaces the anonymous local deployment.
3. Note the two URLs it prints, or find them in the dashboard under Settings > URL & deploy key:
   - the **deployment URL**, `https://<name>.convex.cloud`, which the apps call;
   - the **site URL**, `https://<name>.convex.site`, which receives webhooks.

The first push fails until step 2.4 sets the Clerk variables. That's expected.

---

2. Clerk

1. Create an application at clerk.com named QuillFin. Choose the sign-in methods you want, such as email and Google.
2. **For the web app**: go to Configure > JWT templates > New template > **Convex**. Keep the name `convex`. Copy the **Issuer** URL; this is your Clerk Frontend API URL.
3. **For the Flutter app**: go to Configure > OAuth applications > Add.
   - Name: QuillFin app.
   - Make it a **public** client (PKCE, no client secret).
   - Scopes: `openid profile email`.
   - Redirect URIs: `com.savepoint.quillfin://callback` (Android and iOS) and `http://127.0.0.1:47615/callback` (Windows, Linux and macOS: the app listens there only while signing in).

Copy the **Client ID**.

4. Set both values on Convex:

```
cd backend
npx convex env set CLERK_JWT_ISSUER_DOMAIN https://<your-frontend-api>.clerk.accounts.dev
npx convex env set CLERK_OAUTH_CLIENT_ID <client id>
npx convex dev --once
```

5. **Account deletion** (both app stores require it): set Clerk's secret key (Configure > API keys) on Convex, so `account:deleteAccount` can delete the Clerk user after cancelling Stripe and before wiping the data:

```
npx convex env set CLERK_SECRET_KEY sk_test_...
```

Then turn off Clerk's own **Allow users to delete their accounts** setting, so the web app's account page only offers QuillFin's **Delete my account**: Clerk's button would delete the sign-in but leave the synced data and any Pro subscription behind.

---

3. Anthropic (AI generation on QuillFin's key)

1. Create an API key at console.anthropic.com, in an organisation with billing set up. Set a monthly spend limit there as a backstop to the plan limits.
2. Set it on Convex:

```
npx convex env set ANTHROPIC_API_KEY sk-ant-...
```

To use a different model than the default `claude-sonnet-5-5`, also set `QUILLFIN_AI_MODEL`.

---

4. Stripe

Do this in **test mode** first. Repeat it in live mode before launch.

1. Create a product, **QuillFin Pro**, with two recurring prices: monthly and yearly. Copy both price ids (`price_…`).
2. Go to Settings > Billing > Customer portal and turn it on. Allow cancelling and switching between the two prices.
3. Go to Developers > Webhooks > Add endpoint.
   - URL: `https://<name>.convex.site/stripe/webhook` (the **site** URL).
   - Events: `checkout.session.completed`, `customer.subscription.created`, `customer.subscription.updated`, `customer.subscription.deleted`, `customer.subscription.paused`, `customer.subscription.resumed`.

Copy the signing secret (`whsec_…`).

4. Set everything on Convex. `SITE_URL` is where the web app will live; Checkout returns there.

```
npx convex env set STRIPE_SECRET_KEY sk_test_...
npx convex env set STRIPE_WEBHOOK_SECRET whsec_...
npx convex env set STRIPE_PRICE_PRO_MONTHLY price_...
npx convex env set STRIPE_PRICE_PRO_YEARLY price_...
npx convex env set SITE_URL https://quillfin.savepoint.co.nz
```

---

5. Check it

- In the Convex dashboard, Functions should list `sync`, `billing`, `ai` and `stripeActions`. Settings > Environment variables should list everything above.
- Under Stripe > Webhooks > your endpoint, use "Send test webhook" with `customer.subscription.updated`. It answers 200; the subscription is ignored because it belongs to no QuillFin user, and the Convex logs say so.
- The end-to-end check (sign in, upgrade, sync) needs the apps from Phases 3 and 4.

---

6. The Flutter app

1. Copy `config/example.json` to `config/dev.json` (git-ignored) and fill it in:
   - `QUILLFIN_CLERK_DOMAIN`: the Frontend API host from step 2.2, without `https://`.
   - `QUILLFIN_CLERK_CLIENT_ID`: the OAuth client id from step 2.3.
   - `QUILLFIN_CONVEX_URL`: the deployment URL from step 1.3.
   - `QUILLFIN_SITE_URL`: the web app's URL.

   These are all publishable values. Keys and secrets stay in Convex.
2. Run with `flutter run --dart-define-from-file=config/dev.json`. Release builds take the same flag.
3. Check, on the first real sign-in:
   - The browser opens Clerk, and after signing in the app moves on by itself. On desktop, the tab says "You're signed in to QuillFin".
   - Settings > Sync reaches "Last synced just now", and a note made on a second device shows up after **Sync now**.
   - Leave the app signed in for more than an hour (Clerk ID tokens are short-lived), then sync again. This checks that Clerk's refresh grant returns a new ID token. If it doesn't, sync shows "Your session has expired", and `lib/auth/clerk_oauth.dart` needs to request the token differently.
   - Settings > Plan shows Free with 0 of 20, and **Upgrade monthly** opens Stripe Checkout in test mode (card `4242 4242 4242 4242`). After paying, coming back to the app shows Pro.

7. The web app

1. Copy `webapp/.env.example` to `webapp/.env.local` (git-ignored) and fill it in:
   - `NEXT_PUBLIC_CONVEX_URL`: the deployment URL from step 1.3.
   - `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` and `CLERK_SECRET_KEY`: from Clerk, Configure > API keys. The secret key belongs only in the web host's environment, never in the repo or the browser.

   The Clerk JWT template named `convex` from step 2.2 is what the web app sends to Convex.
2. Run `npm run web:dev` and sign up at http://localhost:3000.
3. Deploy, for example on Vercel:
   - root directory `webapp`, install command `npm ci` (run from the repo root, for the workspaces);
   - the same three environment variables;
   - in Clerk, add the production domain;
   - set Convex's `SITE_URL` (step 4.4) to the web app's URL, since Stripe returns there.
4. Check, on the first real run:
   - Sign up on the web, then sign in to the app with the same account: notes made on either side appear on the other (the app after **Sync now**).
   - Answer a question on the web, then check it isn't due in the app.
   - Settings > Plan: **Upgrade** opens Stripe Checkout, and after paying the page changes to Pro by itself.

Production

`npx convex deploy` creates the production deployment, which has its own environment variables: set them all again there (`CLERK_JWT_ISSUER_DOMAIN`, `CLERK_OAUTH_CLIENT_ID`, `CLERK_SECRET_KEY`, `ANTHROPIC_API_KEY`, the four `STRIPE_*` values and `SITE_URL=https://quillfin.savepoint.co.nz`), with Clerk's production instance and Stripe's live-mode values. Never commit any of these values.

The web app lives at **quillfin.savepoint.co.nz**. Clerk's production instance asks for a few CNAME records on that domain. Add them in Cloudflare as **DNS only** (grey cloud); specific records win over the `*.savepoint.co.nz` wildcard that sends everything else to loganserver. When the Next.js app is deployed, point `quillfin.savepoint.co.nz` at it (a CNAME for Vercel, or a container behind Caddy on loganserver) in place of today's device-only Flutter web build (`hosting.md`).

Store submission (listings, signing, privacy answers) is covered in `store_listing.md`.